SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO1: *Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.* (BL1, BL2)

Activity Tool : Short Answer Text(High)
Assessment Tool Weightage: 40%

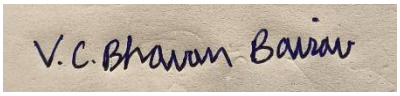
QUESTIONS		
Q. No	**Question**	**Bloom's Level**
1	Explain schema and instance with examples. Distinguish between logical schema, physical schema, and view schema.	BL1 – Remember
2	Identify the entities, attributes, and relationships for a Bank Management System and construct its ER diagram.	BL2 – Understand
3	Explain the Extended ER (EER) model. Describe the concepts of generalization, specialization, and aggregation.	BL2 – Understand
4	Design an EER diagram for a Hospital Management System incorporating inheritance and weak entities.	BL2 – Understand
5	Explain the role of a DBA (Database Administrator) and the responsibilities associated with the position.	BL1 – Remember

---

Code of Conduct:

I V.C.Bhavan Bairav - 192511369 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	30%	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Linkages	25%	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Organization	20%	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Integration of Literature	15%	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity	10%	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ANSWERS

Q1. Explain schema and instance with examples. Distinguish between logical schema, physical schema, and view schema.

A database schema is the overall logical design/structure of a database — the set of relations, attributes, data types and constraints that is defined once when the database is created and changes only occasionally, when the requirements of the organisation change. A schema is like a "blueprint" of a building. For example, the schema for a student database can be written as Student(RollNo, Name, Department, Age), which fixes what attributes every student record will have.

A database instance, on the other hand, is the actual data stored in the database at a specific moment in time — a snapshot of the schema filled with values. Unlike the schema, an instance keeps changing every time a record is inserted, updated or deleted. For example, an instance of the above schema could be the row (101, Aravind, CSE, 19). If a new student is admitted, the instance changes, but the schema (the structure) remains the same.

The ANSI/SPARC three-schema architecture divides a database description into three levels, which helps achieve data independence:

• Physical schema (internal level): Describes how the data is actually stored on the storage device — file organisation, indexing methods, storage allocation and access paths. This level is concerned with hardware/implementation details and is normally handled by the database administrator.

• Logical schema (conceptual level): Describes the structure of the whole database for the community of users — the entities, attributes, relationships, and constraints — independent of how the data is physically stored. Database designers and programmers work mainly at this level.

• View schema (external level): Describes only the part of the database that a particular user or application needs to see, hiding the rest of the database for simplicity and security. Different users can have different views built on the same logical schema.

The key distinction is the level of abstraction: the physical schema is closest to the hardware, the logical schema gives a unified conceptual picture of the entire database, and the view schema is the closest to the end user, tailored to individual requirements. Physical data

independence allows the physical schema to be changed without affecting the logical schema, and logical data independence allows the logical schema to be changed without affecting the external/view schemas.

Q2. Identify the entities, attributes, and relationships for a Bank Management System and construct its ER diagram.

Entities and their key attributes:

- Customer (CustomerID [PK], Name, Address, Phone, Email)
- Account (AccountNo [PK], Type, Balance, OpeningDate)
- Branch (BranchID [PK], BranchName, Location, IFSC)
- Employee (EmployeeID [PK], Name, Designation, Salary)
- Transaction (TransactionID [PK], Date, Amount, Type)
- Loan (LoanNo [PK], Amount, Type, InterestRate)

Relationships and cardinalities:

- Opens: Customer (1) — Account (N). One customer may open many accounts, but every account belongs to exactly one customer.
- Maintained at: Account (N) — Branch (1). Many accounts are maintained at one branch.
- Performs: Account (1) — Transaction (N). One account can have many transactions.
- Works at: Employee (N) — Branch (1). Many employees work at one branch.
- Avails / sanctions: Customer (1) — Loan (N), and Loan (N) — Branch (1).

A customer may avail several loans, each sanctioned by one branch.

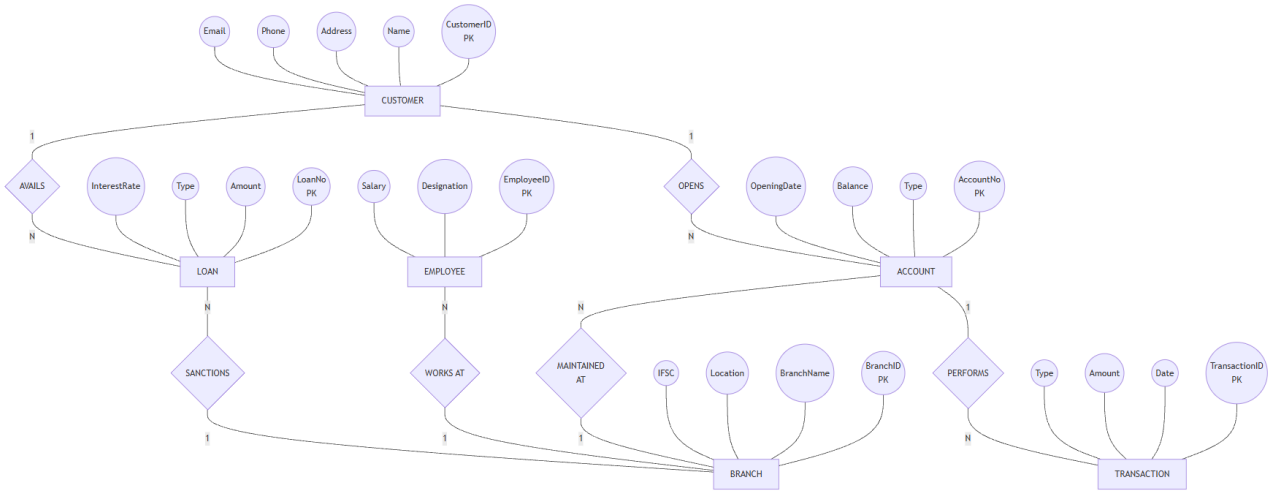


Figure - ER Bank Management System

Q3. Explain the Extended ER (EER) model. Describe the concepts of generalization, specialization, and aggregation.

The Extended Entity-Relationship (EER) model builds on the basic ER model by adding a few more powerful concepts — generalization, specialization, and aggregation — so that complex, real-world applications with hierarchical and composite relationships can be modelled more accurately. It is mainly used as a bridge between conceptual design and object-oriented / relational implementation.

• Generalization: This is a bottom-up process in which two or more lower-level entities that share common attributes are combined to form a single higher-level (generalized) entity. For example, entities SAVINGS_ACCOUNT and CURRENT_ACCOUNT, which both have AccountNo and Balance, can be generalized into a single super-type entity ACCOUNT.

• Specialization: This is the reverse, top-down process, in which a higher-level (super-type) entity is divided into two or more lower-level (sub-type) entities based on some distinguishing characteristic. For example, the entity EMPLOYEE can be specialized into MANAGER, ENGINEER, and CLERK, each having attributes specific to that role in addition to the attributes inherited from EMPLOYEE. Specialization/generalization is commonly shown with an “ISA” (“is-a”) triangle connecting the super-type to its sub-types, and can be total or partial, and disjoint or overlapping.

• Aggregation: This concept treats a relationship between entities as a higher-level entity, so that it can, in turn, participate in another relationship. It is used when a relationship itself needs

to be described further or connected to another entity. For example, the relationship WORKS_ON (linking EMPLOYEE and PROJECT) can be aggregated into a single abstract entity so that it can be related to a MANAGER who monitors that particular employee-project assignment — something that cannot be expressed directly with a simple ER relationship.

Together, these constructs let a designer capture inheritance (via generalization/specialization, analogous to class hierarchies in object-oriented design) and higher-order relationships (via aggregation), making the EER model far more expressive than the basic ER model for complex enterprise applications.

Q4. Design an EER diagram for a Hospital Management System incorporating inheritance and weak entities.

Entities identified:

- Hospital (HospitalID [PK], Name, Location)
- Department (DeptID [PK], DeptName, HospitalID [FK])
- Staff (StaffID [PK], Name, Contact) — generalized super-type
- Doctor (Specialization) and Nurse (ShiftTiming) — sub-types of Staff, related through an ISA (generalization/specialization) hierarchy, so both inherit StaffID, Name and Contact from Staff and add their own specific attributes.
- Patient (PatientID [PK], Name, Age, Address)
- Appointment (ApptID [PK, partial], Date, Time) — depends on Patient and Staff
- Ward-Bed (WardNo [PK, partial]) — a weak entity that cannot exist without its owner entity Department; it is identified only by the combination of DeptID (from Department) and its own partial key WardNo.
- Prescription (PrescNo [PK, partial], Medicine, Dosage) — a weak entity that is existence-dependent on Appointment; a prescription cannot exist without an associated appointment, so it borrows ApptID as part of its identifying key.

Relationships: Hospital has Department (1:N); staff works in Department (N:1); patient books an Appointment with Staff (specifically a Doctor) (N:M realised through Appointment); Appointment generates Prescription (identifying relationship, shown with a double diamond, since Prescription is weak); Department is assigned to Ward-Bed (identifying relationship, since Ward-Bed is weak). In the diagram, weak entities and their identifying relationships are drawn with double-bordered rectangles/diamonds, and total participation of the weak entity in its identifying relationship is shown with a double line.

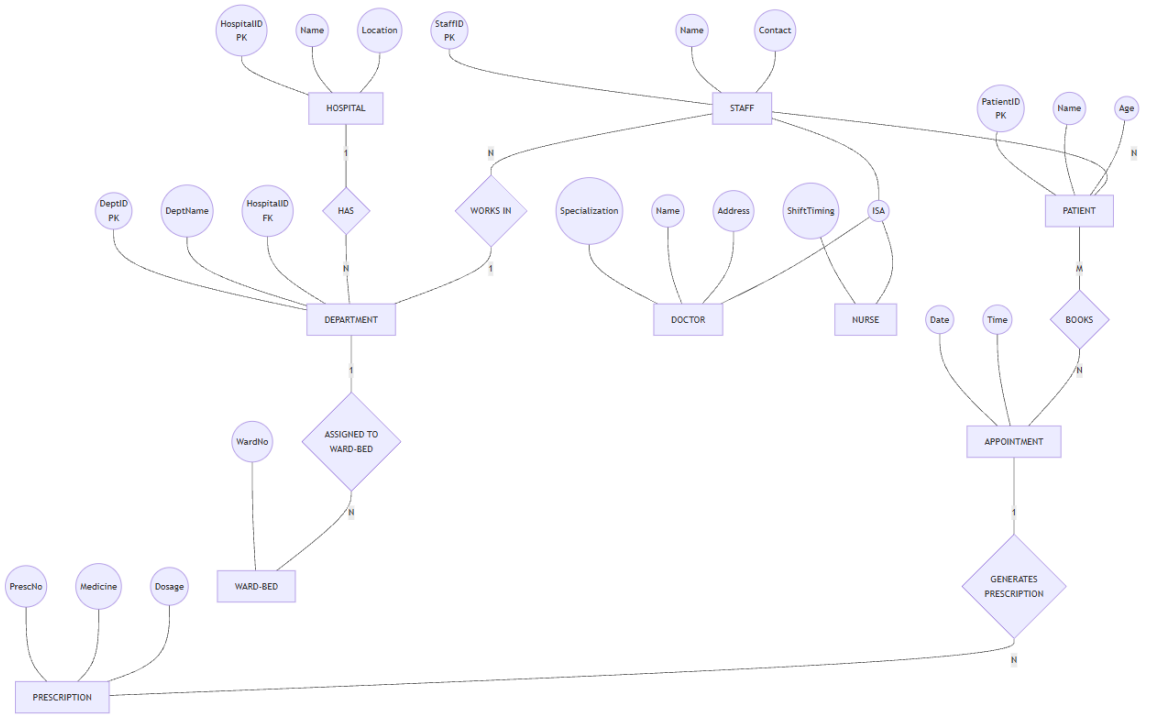


Figure - EER diagram for a Hospital Management System

Q5. Explain the role of a DBA (Database Administrator) and the responsibilities associated with the position.

A Database Administrator (DBA) is the person (or team) responsible for the overall control, management, and maintenance of a centralized database system. The DBA acts as the custodian of the data and ensures that the database remains available, secure, consistent, and efficient for all authorized users. Since the database is a shared corporate resource, the DBA's role is central to keeping the three-schema architecture (physical, logical, and view levels) working smoothly together.

Key responsibilities of a DBA include:

- Schema definition: Creating and modifying the original database schema by writing DDL (Data Definition Language) statements, and deciding the logical and physical structure of the database.
- Storage structure and access method definition: Deciding how data will be represented and stored, and creating the appropriate indexes/access paths for efficient retrieval.

• Granting authorization for data access: Deciding which users/roles can access which parts of the data, and issuing appropriate access permissions to maintain the security of the system.

• Routine maintenance: Taking periodic backups, performing recovery in case of failures, monitoring disk space usage, and applying software patches/upgrades to the DBMS.

• Ensuring data integrity and consistency: Defining and enforcing integrity constraints so that the data stored always remains accurate and consistent with business rules.

• Performance monitoring and tuning: Observing the performance of queries and transactions and reorganizing/tuning the database (indexing, query optimization) to improve efficiency.

• Liaising with users: Working with application developers and end users to understand new requirements, resolve data-related conflicts, and coordinate changes to the schema.

In short, the DBA is responsible for the technical implementation, security, performance, and reliability of the database, ensuring that it continues to serve as an accurate and available resource for the whole organisation.